

Supplementary material

VCF-RDFizer: A Conversion Tool to Represent Variant Data as a Knowledge Graph

Elias Crum, Bart Buelens, Ruben Taelman, Gökhan Ertaylan

This document holds the evaluation’s bookkeeping: how the campaign was run and recorded (S1), the coverage of VCF versions and awkward inputs (S2), configuration and operating-mode coverage (S3), and the defects testing found and how each was resolved (S4), and two measurements the main text reports in its prose (S5). The main text reports the results that bear on its contributions; everything here supports them and none of it changes them.

S1 Reproducibility and execution

Release and image. The tagged benchmark configuration uses VCF-RDFizer v3.1.0 (commit d3b34d5) from `ecrum19/vcf-rdfizer:v3.1.0`, with resolved image digest

`sha256:1904e96dde12ab2e2e70d8ee1267765c293ab100a8bd1b14d5b009b2bf8e34aa`

Image rebuilding is disabled during execution so that all cells in the tagged run use the same container. The per-cell manifests retain the actual commit and image identifiers used for each measurement. v3.1.0 bundles the VCF Core SHACL shapes from vocabulary commit 5bfd19d. The regional-retrieval cells postdate v3.1.0 (they need the regional runner and `tabix`) and were produced by commit 3da1a5a. The use case (Section 4.2 of the main text) converts with the v3.2.0 image and links and governs with v3.3.0’s linkers and policy plug-in v0.2.0.

Pinned toolchain. VCF-RDFizer v3.1.0 fixes its dependencies inside the container image: RMLStreamer 2.5.0, hdtc 1.1.0, pycottas 1.1.0, DuckDB 1.5.5, cyvcf2 0.34.0, pyshacl 0.30.1, Comunica 5.3.0 for N-Triples, @comunica/query-sparql-hdt 5.0.1, the COTTAS Comunica engine 0.1.0, bcftools from the base distribution, and QLever pinned by image digest. `rdf2hdt` and `hdt2rdf` are compiled from the hdt-cpp repository at image build time, so their exact revision is fixed by the image digest rather than by a version number; the resolved image for each benchmark run is recorded with its command and metrics.

Hosts. Experiments run on two physically separate, identically specified hosts (`vcf-bench-1` and `vcf-bench-2`): Ubuntu 24.04.4 LTS, kernel 6.8.0-85-generic, Intel Xeon Silver 4309Y at 2.80 GHz (8 cores), 33.6 GB RAM with swap disabled, a 210 GB SSD, Docker 29.7.2, and Python 3.12.3. Each complete experiment is assigned to one host and executed without concurrent benchmark jobs, keeping comparisons within-host. On a shared calibration workload the two hosts produced the same graph: identical triple sets, a byte-identical COTTAS artifact and HDT index, and N-Triples and HDT files that differ only in line order and in the HDT header’s `dcterms:issued` timestamp. Their speed differed: the conversion phase took 26 s on `vcf-bench-1` and 37 s on `vcf-bench-2`, and validation 156 s and 208 s. Timings are therefore compared only within a host; the calibration characterizes, rather than eliminates, variation between hosts.

What each cell records. The harness creates a fresh output directory for each cell and records its command, host, tool commit, container image, exit status, and wall time. Tool reports provide stage timings, peak resident set size, triple counts, artifact sizes, and validation outcomes; workspace monitoring records transient disk use separately from final artifact size. Input sizes used in performance comparisons are uncompressed VCF bytes, while the corpus table reports

Table S1: The benchmark campaign, grouped by the question each experiment answers. *Recorded*: a non-zero exit that the cell observed without asserting an outcome; *refused*: the cell required the configuration to be rejected, and it was; *skipped*: a guard refusal with a stated reason. Hosts: **vcf-bench-1** (b1), **vcf-bench-2** (b2). Every cell was produced by the published v3.1.0 image. Section numbers prefixed S are in this document; the others are in the main text.

Experiment	Cells	Outcome	Host	Hours	Question answered
<i>Conversion, scaling and storage</i>					
Records scaling	10	all OK	b1	10.99	How cost grows with records (§4.3)
Sample representation	30	29 OK, 1 skipped	b2	19.69	How the two profiles grow with samples (§4.3)
Storage mode	8	all OK	b1	12.87	Peak workspace of plain vs space-optimized (§4.3)
Corpus breadth	10	9 OK, 1 skipped	b2	26.90	HDT and COTTAS on ten public inputs (§4.3)
<i>Input and configuration coverage</i>					
Representation axes	20	all OK	b1	4.82	VCF 4.1–4.5; cost of INFO and header representations (§S2, §S3)
Difficult inputs	16	13 OK, 2 recorded, 1 skipped	b1	0.10	Eleven awkward VCFs and the demonstration linkers (§S2)
Covering set	10	all OK	b1	0.35	Every option value and pair (§S3)
Modes	7	all OK	b1	0.03	Each operating mode on its own (§S3)
Feasibility	9	all OK	b1	9.03	Memory ceiling $\times$ configuration (§S3)
<i>Validation and retrieval</i>					
Robustness	9	all OK	b1	0.25	Determinism, round trip, mutation score (§4.1)
Equivalence	8	6 OK, 2 refused	b2	6.84	Four engines, both profiles, three artifacts (§S3)
Query cost	6	all OK	b1	4.27	SPARQL vs cyvcf2 on identical work (§4.4)
Total	143	136 OK, 2 recorded, 2 refused, 3 skipped		96.16	

compressed download sizes. Replicated comparisons report variability or confidence intervals; single-run corpus measurements describe the tested input and configuration without estimating run-to-run variation. The per-cell records, normalized query results, and comparison reports are archived in the testing repository.

The campaign. The campaign comprises 143 recorded cells over 96.2 h of measured wall time (Table S1). Every cell records its own argv, host, tool commit and image digest, and the integrated record of all cells is archived with the testing repository. A non-zero exit is classified by what the cell asserted: two cells were required to be refused and were (*refused*), two recorded an exit status without asserting an outcome (*recorded*), and three were skipped by a stated guard. No cell failed.

Chunked construction of HDT and COTTAS. The representation stage reads the plain or gzip aggregate incrementally, respects newline-terminated N-Triples record boundaries, and materializes one raw chunk in a Docker-managed workspace. Each selected representation builder consumes that chunk before it is deleted and the next one is created (Figure S1). At most one uncompressed RDF chunk exists at a time; the final parts, merge intermediates, indexes and converter scratch space still require storage, but no full decompressed duplicate of the aggregate is created. HDT chunks are converted with **rdf2hdt** from hdt-cpp [1, 2] and merged pairwise in a balanced tree with **hdtc create** [3], which performs the same logical merge as HDTCat [4] under a fixed memory budget and without Java; an unpaired file is carried into the next round,

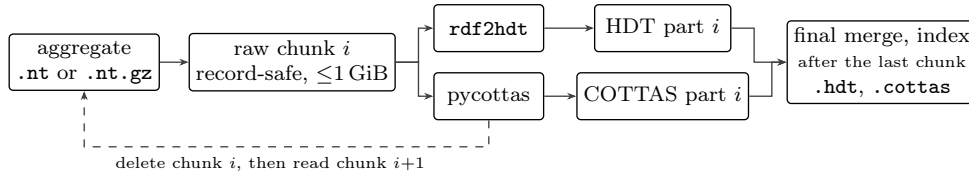


Figure S1: One chunk at a time: each record-safe chunk is consumed by every selected builder and deleted before the next is read.

Table S2: Public VCF corpus. File names are canonical `.vcf.gz` stems; later tables abbreviate the GIAB identifiers. Assembly and sample counts were verified from each file’s header. Sizes are approximate compressed download sizes; the tool processes the uncompressed stream.

File	Provider	Assembly	Samples	MB (gz)	Evaluation role
NG1N86S6FC	Sequencing.com	GRCh38	1	379	Consumer WGS
NG131FQA1I	Dante Labs	GRCh38	1	224	Consumer WGS
NB72462M	Nebula Genomics	GRCh38	1	341	Consumer WGS
60820188475559	PGP upload	GRCh37	1	325	SNP-only profile
60820188474283	Dante Labs	GRCh37	1	222	Consumer WGS
OG00R_HG002	precisionFDA	GRCh38	1	69	GIAB Truth Challenge v2
1000G_phase3_chr20	1000 Genomes	GRCh37	2,504	327	Phased cohort
HGSVC2	HGSVC2	GRCh38	32	31.5	Multi-sample SVs
HG004_GRCh38	GIAB v4.2.1	GRCh38	1	149	Benchmark genome, chr1–22
HG005_GRCh38	GIAB v4.2.1	GRCh38	1	139	Benchmark genome, chr1–22

and each merge’s scratch directory is removed as soon as that merge completes. The final HDT is indexed by `hdtc`, producing the sibling `.hdt.index.v1-1` index without executing a query; intermediate indexes are not retained. COTTAS chunks are converted with `pycottas` [5] and merged in one streaming k -way pass rather than pairwise, so no global in-memory hash table or full-size temporary spill is built. In v3.1.0 the merged output carries a fixed `spo` index; v3.2.0 adds the other index orders. Each COTTAS operation runs in an isolated temporary working directory, so that its local DuckDB scratch database cannot be reused by a later chunk or merge. Chunk conversion, merging, final indexing, validation, and packaging are timed separately.

Chunk sizes. By default a chunk targets 512 MiB of uncompressed N-Triples, bounded below by 128 MiB and above by 1 GiB (`-chunk-target-bytes`, `-chunk-min-bytes`, `-chunk-max-bytes`). For constrained workspaces the documented lower-peak setting is 128 MiB target, 64 MiB minimum and 256 MiB maximum, trading additional chunks and merges for a lower temporary peak. Every experiment used the defaults except six cells of the memory-feasibility sweep, which used the lower-peak setting (S3).

Distribution and maintenance. Each tagged VCF-RDFizer version is published across PyPI, Conda-Forge, and Docker Hub, so users can choose pip-, conda-, or container-based installation with matched versions, under the MIT License [6]. The GitHub README is the primary user guide: quick-start commands, end-to-end examples, command-line option documentation, and troubleshooting notes. The project follows a release-oriented workflow with versioned release notes, public issue triage, and a unit-test suite that runs on Linux, macOS and Windows.

S2 Input and version coverage

The public corpus. Table S2 lists the ten public VCFs. Download commands and canonical filename definitions are maintained in the testing repository.

Declared VCF versions. Six single-purpose fixtures covering VCFv4.1 through VCFv4.5, plus one file with no `##fileformat` declaration, all converted with exit code 0. The generated

Table S3: Difficult-input fixtures. Both “converted” and “refused with a clear diagnostic” are acceptable outcomes; a crash or a silently wrong graph is not, and neither occurred. Triple counts are for the complete graph the run produced.

Fixture	Outcome	Triples	Validation	Note
CRLF line endings	converted	262	PASS	No stray <code>\r</code> in the last field’s literal
Symbolic alleles	converted	453	PASS	<code></code> not treated as a base string
Breakend ALT	converted	417	PASS	Breakend ALT syntax preserved
Mixed ploidy	converted	350	PASS	Ploidy 4 then 3 followed the data
Half and haploid calls	converted	446	PASS	Four genotype cases stayed distinct
Missing values	converted	321	PASS	Missing QUAL, FILTER, INFO and FORMAT absent from the graph
FILTER variants	converted	486	PASS	PASS, . and a failed code are three states
Declared FORMAT unused	converted	293	PASS	Header declaration represented although never used
Zero data records	converted	189	PASS	Header-only input; no data records
Sites only	converted; validation not completed	245	incomplete	Graph and HDT artifact produced
Truncated gzip	refused at decompression	—	not reached	Refused in 0.58 s with a decompression diagnostic; intended

RML mapping for each version is byte-identical apart from the asserted file class (`vcfc:VCF41File` through `vcfc:VCF45File`), which locates version awareness in the typed file resource rather than in field-level rules. Field handling is therefore data-driven: the VCFv4.5 fixture’s local-allele **FORMAT** keys (**LAA**, **LAD**) and the VCFv4.4 fixture’s phase-set key (**PS**) are carried through as ordinary **FORMAT** entries, raising the per-record triple yield from 210 (4.4) to 252.5 (4.5) on otherwise matched two-record fixtures. This is preservation, not interpretation: nothing in the graph resolves a local allele index to a global allele, so VCFv4.4/4.5 support is lexical carry-through pending a dedicated local-allele semantics layer. The undeclared-version file also converted, via the documented fallback.

Real cohorts. Nine of ten corpus files converted end to end, with HDT and COTTAS built and decode-checked for each. The tenth, the 2,504-sample 1000 Genomes chromosome-20 call set, was skipped by the benchmark harness rather than refused by the tool: the harness withholds any corpus file with more than 1,000 sample columns from the expanded arm, because the expanded representation emits one call resource per record and sample. Its full sample dimension was exercised instead through 10,000-record slices carrying all 2,504 samples, which converted in both representations; Section 4.3 of the main text quantifies why the expanded arm cannot be run on the whole file at this disk budget. The tool also applies a workspace check: before converting, it estimates the expanded workspace from the record and sample counts and refuses the run when that estimate exceeds three quarters of the free space, naming the condensed representation as the alternative.

Difficult inputs. Eleven adversarial fixtures were run (Table S3; fixture files are named `awkward_*` in the testing repository). Nine converted and passed validation, including CRLF line endings, symbolic alleles, breakend ALT syntax, ploidy 4 and 3 in the same file, half-calls and haploid calls, missing values in every optional field, header keys declared but never used, the three-way **PASS**/. /failed-filter distinction, and a header-only file with no data records. Two cells exited non-zero. One is a refusal to convert: the truncated gzip stream, rejected in 0.58 s at the decompression stage with a clear diagnostic, which is the intended behaviour. The sites-only file produced 245 triples and an HDT artifact, but its paired validation was not completed (S4).

S3 Configuration coverage, operating modes and memory feasibility

Covering set. A ten-row covering set exercised every value of all seven configuration axes — sample representation, INFO representation, storage mode, RDF compression, representations, artifact compression and HDT strategy — and all ten rows exited 0 and passed validation, including the shapes. Pairwise coverage is complete except for three combinations the wrapper structurally refuses (`-hdt-strategy single` beside space-optimized storage, beside COTTAS, and beside `hdt,cottas`), so the remaining gaps are specification, not omission. Gzip and Brotli were exercised as values of both compression axes; the campaign does not compare the two codecs' ratios or throughput.

Operating modes. Each of the tool's operating modes — `tsv`, `compress`, `decompress`, `index` for HDT and for COTTAS, a conversion-only phase and a validation-only phase — ran independently and exited 0, confirming that conversion and validation are separable stages rather than one monolithic command.

Memory feasibility. A feasibility sweep crossed three memory ceilings (8, 16 and 31 GB) with three configurations on a one-million-record input. All nine cells completed; peak resident memory was 1.34–1.93 GB in every case, below every requested ceiling, which is consistent with the flat memory scaling in Section 4.3 of the main text. This experiment carries one caveat that must be reported with it: the ceiling was requested through Docker memory limits but the harness could not confirm that the pinned release forwards them to its container invocations, so these cells establish that the workload is memory-light, not that it survives a genuinely enforced cap.

The cost of the INFO and header representations. On the structural-variant file, structured INFO produced 96.75M triples against 89.01M for raw INFO (+8.7%) and a 7.4% larger artifact, and on the 100,000-record HG005_GRCh38 input it produced 17.10M against 12.52M (+36.6%). Raw INFO still carries the allele layer here, because the expanded sample profile references it (S4); before that fix the raw arm omitted it and the difference looked larger than it is. Structured header representation costs almost nothing by comparison (17,098,746 against 17,094,293 triples, +0.03%).

Cross-encoding equivalence. The equivalence experiment converted one 10,000-record, multi-sample fixture in both sample representations and both storage modes, built HDT and COTTAS from each, and ran the full twenty-seven-query suite (the fourteen preflight checks and Q1–Q13) under all four engines against all three artifacts. All sixteen validations passed with all four engines in agreement, on graphs of 0.96M triples (expanded) and 0.79M triples (condensed). The complete condensed cell took 70.4 min end to end. Two further cells asserted that the tool must refuse `-hdt-strategy single` beside COTTAS and beside space-optimized storage, and it did, with exit status 2.

S4 Defects found during testing, and their resolution

Testing identified problems in measurement, conversion, and validation. Three of them were conversion defects that the validation layer itself caught; the main text summarizes them in Section 4.1 of the main text. None alters a measurement reported in the main text.

Header-only input omitted its declared samples. A VCF with a declared sample column but no data records produced a graph without the `vcfc:SampleSet` and `vcfc:VCFSample` resources declared by its header. The emitters derived the file IRI from a source name learned from the first data row. The predicate and class censuses (Q9 and Q10) detected the missing resources. Emitting the sample set from the header increased the graph from 181 to 189 triples and allowed validation to complete. The same test exposed an incorrect oracle expectation: `preflight_sample_gt_inventory` expected one `vcfc:sampleId`, although that property belongs

to a per-record `vcfc:SampleCall`. The oracle was adjusted to expect no such calls when the file contains no records.

Allele references without corresponding resources. Applying the shapes to the configuration covering set identified three failing rows, all combining raw INFO with expanded samples. Allele resources were emitted only for structured INFO, but expanded genotypes referenced them through `vcfc:calledAllele` regardless of the INFO setting. A 1,000-record test produced 1,972 such references without the corresponding allele descriptions. The comparison queries counted those edges without following them, whereas the shapes reported 5,916 violations. The emitter was changed to produce the allele layer when either representation requires it. The census oracle also needed the same dependency rule: its initial expectation reproduced the emitter’s assumption and rejected the added triples. After both were updated, all ten covering-set rows passed. Each affected row gained 10,846 triples, while the other seven outputs remained byte-identical. Regression checks cover all four INFO/sample representation combinations and require every referenced allele to be described.

A withdrawn file’s header remained in every release view. During development of the policy plug-in, every record decision was correct, but a withdrawn file’s header and sample set remained in every view, which the record counts alone could not show. The check against the VCF-text oracle caught it. That defect is why the plug-in defines what a withheld resource takes with it by an ownership rule rather than by a list of record subtrees (Section 2.4 of the main text).

Two memory guards that measured the wrong quantity. Extending retrieval to 171M triples (Section 4.4 of the main text) exposed two defects, neither visible at fixture scale. First, the shape layer is size-gated because `pyshacl` is in-memory, but the gate weighed the *packaged artifact* while the cost it guards against is set by the *graph*. One 170,935,101-triple graph therefore fell on both sides of a single 512 MiB threshold according only to how it was packaged: as COTTAS (390.7 MB) it passed the gate and the run was killed at 32.2 GB resident on a 31 GB machine, while the same graph as gzip-framed N-Triples (756.6 MB) or HDT (1.18 GB) was skipped. The better a format compressed, the likelier it was to exhaust memory. The gate now counts triples, which no packaging changes. The same gate admitted a 58.2M-triple real-genome graph, a 277 MB artifact, to `pyshacl` during this study’s validation of NG131FQA1I (Section 4.1 of the main text); the host ran out of memory, and that validation was rerun without shapes. Second, the Comunica-backed endpoints aborted with a JavaScript heap exhaustion while roughly 25 GB of the host’s memory was free, because Node does not size its heap from the machine and no ceiling was set; the ceiling is now configurable. Both corrections are released in v3.3.0 and neither alters conversion output; the failing runs are archived alongside the successful ones.

COTTAS query execution and timeout enforcement. The initial COTTAS query implementation used `pycottas`’ RDFLib store. On the condensed representation it completed the fourteen preflight queries and Q1–Q4, then failed to complete `q05_sample_genotype_counts`. Two attempts ran for approximately 41 h and 10 h at roughly 190% CPU before being stopped. The second had a 1,800 s per-query timeout and a 14,400 s validation budget, but neither interrupted the in-process query. Both runs are archived. Replacing this path with the DuckDB-backed Comunica engine reduced Q5 to 2–5 s and the thirteen-query workload on the 0.96M-triple comparison graph to 38 s, against 1,402 s through RDFLib. Query execution now occurs where the timeout can terminate it, and the completed cross-engine measurements use this engine.

SHACL report interpretation. Integrating the vocabulary’s SHACL shapes exposed two reporting errors. The validator treated `pyshacl`’s `conforms` flag as a blocking verdict even for recommendations, causing conformant INFO declarations without `Source` to be rejected. Its violation matcher also expected a report format that `pyshacl` did not emit. The parser was corrected to classify results by severity, with only `sh:Violation` blocking a run.

Missing-token preflight false positives. The preflight reports contain 608 checks: 599 passed and nine flagged the missing-token rule, all on the 100,000-record HG005_GRCh38 input.

Table S4: Retrieval at scale, on graphs built from HG005_GRCh38 by the published v3.1.0 image. Times are for the thirteen core questions of Table 3, excluding the one-time index build, which is reported separately. Every completed question’s answer was compared against cyvcf2 before its timing was read. The Comunica-backed engines do not complete the workload at this size; their totals cover the five questions they answered, and are not comparable with a thirteen-question total. The three packagings of the whole-genome graph differ by 0.3% in query time; what differs between them is the one-time decode, reported in the last column.

Graph	Triples	Engine	Time (s)	Outcome
<i>Does SPARQL scale to a whole genome? (all thirteen questions)</i>				
Fixture	958,919	QLever	1.1	1.15 s per million triples
100 k records	17,098,746	QLever	17.0	0.99 s per million
1 M records	170,935,101	QLever	163.3	0.96 s per million
Whole genome	657,425,805	QLever	634.6	0.97 s per million; setup 868.0 s; replicates within 0.7%
<i>Does the artifact matter? (whole genome, one engine, three packagings)</i>				
from N-Triples	gzip-framed		634.6	Decode 179–192 s; setup 868.0 s
from HDT	—		636.3	Decode 955–972 s; setup 861.8 s
from COTTAS	—		636.6	Decode 592–601 s; setup 868.4 s
<i>Do the other engines get there at all? (1 M records, 170,935,101 triples)</i>				
COTTAS engine	5 of 13 answered		743.1	Native allocation failure on Q6
HDT engine	5 of 13 answered		12,432.2	Reached the one-hour per-query ceiling on Q5

The rule looks for bare "." literals that should have been typed as `vcfc:Null`. It first returned 20 rows there, all on `vcfc:fieldNumber` or `vcfc:genotypeString`, where a dot is legitimate: `Number=.` denotes variable cardinality, and a fully missing genotype is written `..`. The rule was narrowed to exclude those two predicates. In v3.1.0 it still returns ten rows, and they are the same case one layer over: the header declares ten INFO fields with `Number=.`, and the header-attribute layer also carries each declaration’s Number attribute verbatim as a `vcfc:attributeValue` of ".". A follow-up change excludes that attribute’s value as well while still reporting a bare dot on any other attribute; it postdates v3.1.0, so the ten flags remain in this campaign’s reports. None of the flags reflects malformed output.

Sites-only fixture limitation. The sites-only fixture produced 245 triples and a valid HDT artifact, but its two validations ended with `EXECUTION_FAILED`. The fixture declared a `FORMAT` column without sample columns, which `bcftools` rejects. This is a fixture/oracle limitation rather than evidence that the generated graph failed semantic comparison. No completed paired result is claimed for this input, and a conformant sites-only fixture remains necessary to establish that coverage.

Measurement and reporting in the legacy benchmark. The earlier six-file, batch-oriented benchmark exposed two measurement problems. Partitioned compression wall times were under-reported; the timing implementation was corrected in commit `fb48e50`, and the affected historical durations were reconstructed from wrapper logs and execution timestamps. In addition, an input-size column labelled `.vcf.gz` actually contained the uncompressed VCF stream size: the retained `NG131FQA1I.vcf` contains 1,215,509,730 bytes, matching the reported 1215.5 MB. The compression denominator also affected interpretation: the legacy HDT reduction of approximately 92–95% was measured against uncompressed N-Triples, whereas in the current corpus experiment HDT is 1.36–1.82 times the size of the stored gzip-framed N-Triples. These use different reference artifacts and are not a like-for-like change in compression performance, so the legacy conversion and compression tables are excluded from the current analysis.

S5 Additional measurements

Figure S2 and Table S4 hold measurements that Section 4.3 of the main text and Section 4.4 of the main text report in the text.

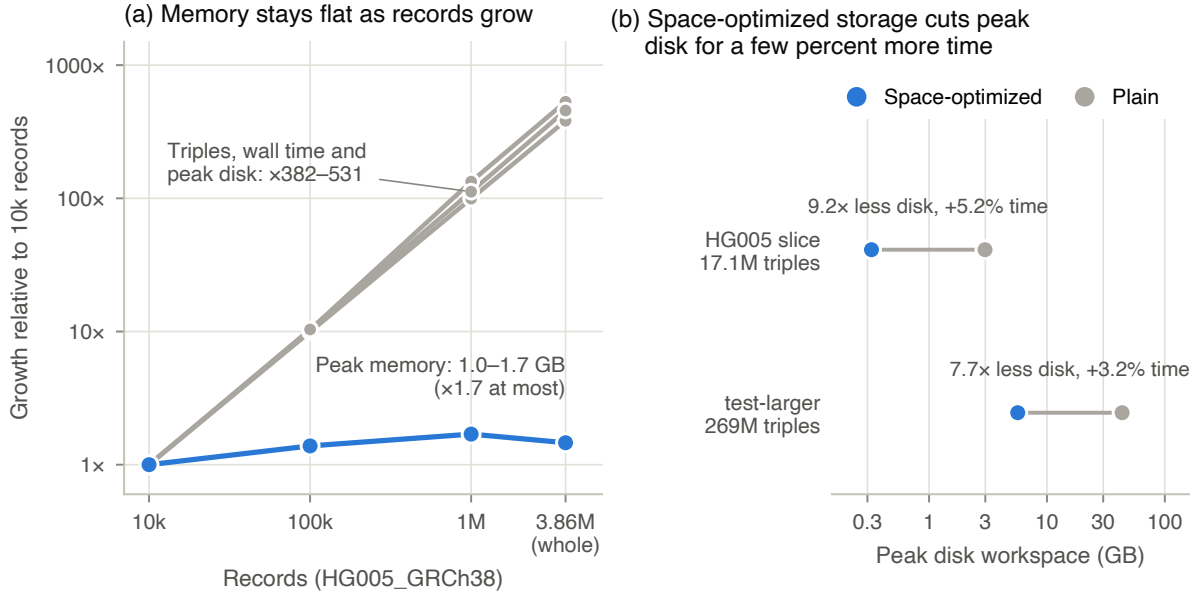


Figure S2: **Record scaling and storage mode.** (a) Growth of each measure relative to the 10,000-record rung of the HG005_GRCh38 ladder (medians of three replicates; the whole file is one run). Triples, end-to-end wall time and peak disk workspace grow in proportion to the records, while the mapping stage’s peak resident memory stays between 1.0 and 1.7 GB. (b) Peak disk workspace of the plain and space-optimized storage modes on two inputs, with the end-to-end time cost of space-optimized mode. Final artifacts and triple counts are identical in both modes. Both panels are from the v3.1.0 run on `vcf-bench-1`.

References

- [1] RDF/HDT contributors. `hdt-cpp`: HDT C++ library and tools, 2026. Built from the repository head at image build time; the exact revision is fixed by the VCF-RDFizer v3.1.0 image digest.
- [2] Javier D. Fernández, Miguel A. Martínez-Prieto, Claudio Gutiérrez, Axel Polleres, and Mario Arias. Binary rdf representation for publication and exchange (`hdt`). *Web Semantics: Science, Services and Agents on the World Wide Web*, 19:22–41, 2013.
- [3] FRINK OKN. `hdctc`: HDT creator, 2026. Computer software, MIT License.
- [4] Dennis Diefenbach and José M. Giménez-García. HDTCat: Let’s make HDT generation scale. In *The Semantic Web – ISWC 2020*, volume 12507 of *Lecture Notes in Computer Science*, pages 18–33, Cham, 2020. Springer.
- [5] Julián Arenas-Guerrero. `pycottas`: Python cottas library for compressing and querying rdf, 2026. RDFLib store backend, triple-pattern interface, and COTTAS conversion utilities.
- [6] Rafael C. Jiménez, Mateusz Kuzak, Monther Alhamdoosh, Michelle Barker, Bérénice Batut, Marc Borg, Salvador Capella-Gutiérrez, Neil P. Chue Hong, Martin Cook, Manuel Corpas, Michael Flannery, Leyla Garcia, Josep L. Gelpí, Simon Gladman, Carole Goble, Miguel González Ferreiro, Alejandra Gonzalez-Beltran, Philip C. Griffin, Björn Grüning, Jon Hagberg, Petr Holub, Rob Hooft, Jon Ison, Sarah Jones, Florian Kaiser, Sunghwan Kim, Matthew Krafczyk, Anna Krystalli, Hilmar Lapp, Ann Lonie, Céline Marek, Michael P. Mazanetz, Aleksandra Nenadic, Wolfgang Petersen, Esther Plomp, Mikhail Pomazkov, Karthik Ram,

Stian Soiland-Reyes, Alexey Sokolov, Andreas Struck, Radoslaw Sucheki, Rutger Vos, Alan R. Williams, Patrick Brack, Marta W. Muelas, Alexander D. Diehl, Jean-Baptiste Poline, Teresa K. Atwood, Alexander Mansurov, Larisa N. Soldatova, Carl Kingsford, Paul J. Kersey, Mark D. McDowall, Magne Østeras, Cristina Guijarro-Clarke, and Steve Crouch. Four simple recommendations to encourage best practices in research software. *F1000Research*, 6:876, 2017.